Technical Evaluation

AWS CodeBuild vs GitLab CI (Self-Hosted)

Dynamic CI/CD Pipeline Implementation for AI Code Converter Tool

Senior Project Technical Report

February 2026

Vitaliy Sviridyuk

Executive Summary

Project Constraints

- **Timeline:** 4 months remaining until demo and graduation (May 2026)
- **Infrastructure:** Existing EKS cluster with single t3.small node (2GB RAM), comprehensive Terraform-managed infrastructure
- **S3 Structure:** Existing multi-tenant path structure `s3://aiconverterbucket/users/{user}/conversations/{conversation_id}/generated/`
- **Usage:** ~100 builds/month (conservative estimate), ~10-20 builds/month realistic steady state
- **Dynamic Requirements:** AI-generated build specifications stored alongside converted code in existing S3 conversation paths
- **Sponsor Requirement:** Document portability path to sponsor's self-hosted infrastructure
- **Existing Capabilities:** Presigned URL automation for S3, Terraform IAM management patterns, established EKS networking

Recommendation: AWS CodeBuild

AWS CodeBuild is the recommended platform for this project.

The requirement for **dynamically generated build specifications** stored in the existing conversation-based S3 structure (`users/{user}/conversations/{conv_id}/generated/`) makes CodeBuild's native S3 path-based integration ideal. CodeBuild can directly target the specific conversation directory where the AI has stored both the converted code and its corresponding `buildspec.yml`.

GitLab CI would require extracting these files from their natural location to inject into the runner, adding unnecessary abstraction.

Factor	AWS CodeBuild	GitLab CI (Self-Hosted)
S3 Path Structure	Native support for deep paths	Requires wrapper scripts
Dynamic Buildspec	Direct retrieval from conversation path	Complex extraction needed
Total Implementation	3 sprints (6 weeks)	4-5 sprints (risk of non-delivery)
Monthly Cost	~$13	~$67 (t3.large required)
4-Month Project Cost	**$52**	**~$268**

---

1. Feature Comparison

1.1 AWS CodeBuild

AWS CodeBuild is a fully managed continuous integration service. It integrates naturally with the existing multi-tenant S3 structure.

Dynamic Buildspec Capability: CodeBuild supports S3 sources with path overrides, enabling per-conversation builds:

- Backend triggers CodeBuild with `sourceLocationOverride` pointing to `aiconverterbucket/users/{user}/conversations/{conversation_id}/generated/`
- CodeBuild retrieves the AI-generated `buildspec.yml` from that specific conversation directory alongside the converted source code
- Supports the existing data retention and organization patterns (user isolation, conversation history)

Implementation Characteristics:

- Direct boto3 trigger with dynamic S3 paths following existing structure
- Native S3 integration accepts pre-generated buildspecs stored in conversation directories
- Serverless scaling (20 concurrent builds default)
- Zero infrastructure management

1.2 GitLab CI (Self-Hosted)

GitLab CI runners require additional logic to access dynamic configurations stored in deep S3 paths outside the repository structure.

Dynamic Configuration Challenges:

- Runner must fetch `buildspec.yml` from `s3://bucket/users/{user}/conversations/{id}/generated/` before execution
- Requires either:
  - Custom initialization scripts in every job to `aws s3 cp` the config
  - Backend committing generated configs to a Git repository (breaking the conversation-based storage model)
- Additional latency and complexity for accessing the existing S3 structure

1.3 Feature Matrix

Feature	AWS CodeBuild	GitLab CI (Self-Hosted)
Existing S3 Structure	Native path access	Requires wrapper scripts
Per-Conversation Configs	Native support	Complex extraction needed
Multi-tenancy Isolation	S3 path-based	Requires custom handling
Total Implementation Time	**3 sprints (72 hours)**	**4-5 sprints (100+ hours)**
Sprint 1 Deliverable	Working build pipeline	Infrastructure provisioning only
Risk of Non-Delivery	Low	**High** (may miss demo deadline)
Monthly Cost	~$13	~$67

---

2. Cost Analysis

2.1 AWS CodeBuild Pricing

For 100 builds/month at 10 minutes average duration (1,000 build minutes):

Compute Type	vCPU	Memory	Price/Minute	Monthly Cost
general1.small	2	3 GB	$0.005	$5.00
general1.medium	4	7 GB	$0.010	$10.00
general1.large	8	16 GB	$0.020	$20.00

Recommended: general1.medium ($10/month).

Additional Costs:

- ECR Storage (~2GB): ~$3/month
- S3 requests (List/Get): ~$0.50/month
- **Total Monthly: ~$13**
- **4-Month Project: ~$52**

2.2 GitLab CI (Self-Hosted) Pricing

Cost Breakdown (t3.large dedicated node):

Component	Monthly Cost	4-Month Total
EC2 t3.large (On-Demand)	$60.74	$242.96
EBS gp3 (20GB)	$1.60	$6.40
Data Transfer	~$2.00	$8.00
ECR Storage	$3.00	$12.00
Total	**~$67**	**~$268**

2.3 Security & Isolation Evaluation

Case A: Untrusted code (most sandbox use cases)
The code being built may be malicious, the risks include:

- Exfiltrating AWS credentials / environment variables
- Exfiltrating other users' data
- Resource abuse
- Network scanning

- Privilege escalation

Case B: Trusted repo code (typical CI)
If builds only run on code reviewed in your repo, security is easier:
- Mainly protect secrets + reduce blast radius
- Isolation is less strict = more convenient

For our project, compiling & running converted code per conversation should assume untrusted by default.

We must treat sandbox jobs as untrusted workloads. The system must prevent cross-user data access and limit AWS/network blast radius even if the job code is malicious.

2.31 Execution isolation

AWS CodeBuild
- Runs builds in ephemeral isolated environments (container)
- Runnable CodeBuild inside a VPC with tight network rules
- You can attach an IAM role per project
- Less 'host exposure' due to operating the underlying runner host

GitLab CI (self-hosted runners)
- Runner executor options:
    - Shell executor = worst
    - Docker executor = fair
    - Kubernetes executor = best option for untrusted builds, but more complex than needs to be

- With Docker executer, 'escape' risk and secret leakage depends heavily on config
    - Privileged mode, docker in a docker, mounting docker sock is **dangerous**

- We would be responsible for patching OS, Docker, runner versions, and hardening

2.4 Secrets handling

CodeBuild
- Supports retrieving secrets from **Secrets Manager**
- Should avoid long-lived keys, preferably IAM role credentials via build service
- Control log retention in CloudWatch

We don't wanna inject long life AWS keys into sandbox jobs. Jobs assumed an IAM role or used short lived credentials scoped to a least privileged role (gitlab). Logs must not contain secrets

---

3. Three-Sprint Implementation Timeline

Sprint 1: Foundation (Weeks 1-2)

Goal: AI generates custom buildspecs stored in conversation paths; CodeBuild executes builds directly from `users/{user}/conversations/{id}/generated/`.

Implementation:

Task	Duration	Notes
AI prompt engineering for buildspec generation	6 hours	Generate buildspec.yml based on detected tech stack in conversation
Update Terraform for CodeBuild IAM	2 hours	Following existing `aws_iam_policy_document` patterns
Deploy CodeBuild Project (S3 source mode)	2 hours	Configure for dynamic source locations
Update backend to store buildspec in conversation path	3 hours	Save alongside generated code in `.../generated/buildspec.yml`
Trigger integration with conversation paths	3 hours	Pass full S3 prefix to CodeBuild
End-to-end test	4 hours	Verify buildspec retrieval from deep paths

Sprint 1 Total: 20 hours
Deliverable: Working pipeline that builds Docker images from AI-generated code

S3 Storage Pattern:

```
None
s3://aiconverterbucket/
└── users/
    └── {user_id}/
```

```
└── conversations/
    └── {conversation_id}/
        └── generated/
            ├── src/                    # Converted source code
            ├── buildspec.yml           # AI-generated CI config
            └── metadata.json           # Stack detection results
```

Sprint 2: EKS Integration (Weeks 3-4)

Goal: Generated buildspecs include EKS deployment; backend tracks builds per conversation.

Task	Duration	Notes
Extend AI prompt for K8s deployment	4 hours	Add kubectl stages to generated buildspec
Implement conversation-scoped Job naming	4 hours	Ensure K8s jobs are named per conversation_id for tracking
Backend build status tracking	4 hours	Store CodeBuild build_id in conversation metadata
Frontend integration	8 hours	Show build status in conversation UI
Integration testing	4 hours	End-to-end testing with sandbox deployments

Sprint 2 Total: 24 hours
Deliverable: Full pipeline from code generation to running EKS sandbox

Sprint 3: Polish & Documentation (Weeks 5-6)

Task	Duration	Notes
Buildspec validation	4 hours	Validate AI-generated YAML before S3 upload
Error handling and retry logic	4 hours	Robust failure handling

Task	Duration	Notes
Sponsor migration doc	8 hours	GitLab CI equivalent using same S3 structure
Demo preparation	8 hours	Multi-stack demonstration, documentation

Sprint 3 Total: 24 hours
Deliverable: Production-ready pipeline with sponsor handoff documentation

Total Implementation: 68 hours (~3 sprints)

---

4. Integration Complexity

4.1 Dynamic Buildspec Flow with Existing S3 Structure

Architecture:

1. **AI Conversion:** User submits legacy code in conversation context
2. **Storage:** AI generates `buildspec.yml` → Backend stores in `s3://aiconverterbucket/users/{user}/conversations/{conv_id}/generated/buildspec.yml`
3. **Trigger:** Backend calls `codebuild.start_build()` with source location set to the conversation's generated directory
4. **Execution:** CodeBuild retrieves both source code and buildspec from the conversation path

4.2 Boto3 CodeBuild Integration

Basic Trigger with Dynamic S3 Path:

```python
import boto3

def trigger_build(bucket: str, s3_prefix: str, conversation_id: str) -> dict:
    """
    Trigger CodeBuild with source from specific conversation path.

    Args:
```

```
        bucket: S3 bucket name (aiconverterbucket)
        s3_prefix: Full path to conversation directory
                   (users/{user_id}/conversations/{conversation_id}/generated)
        conversation_id: Unique identifier for this build

    Returns:
        dict containing build_id and initial status
    """
    client = boto3.client('codebuild')

    response = client.start_build(
        projectName='ai-converter-dynamic-build',
        sourceLocationOverride=f'{bucket}/{s3_prefix}',
        environmentVariablesOverride=[
            {
                'name': 'CONVERSATION_ID',
                'value': conversation_id,
                'type': 'PLAINTEXT'
            },
            {
                'name': 'S3_PREFIX',
                'value': s3_prefix,
                'type': 'PLAINTEXT'
            }
        ]
    )

    return {
        'build_id': response['build']['id'],
        'status': response['build']['buildStatus']
    }


def get_build_status(build_id: str) -> dict:
    """Retrieve current status of a build."""
    client = boto3.client('codebuild')

    response = client.batch_get_builds(ids=[build_id])

    if not response['builds']:
        return {'error': 'Build not found'}

    build = response['builds'][0]
    return {
```

```
        'build_id': build_id,
        'status': build['buildStatus'],
        'current_phase': build.get('currentPhase'),
        'start_time': build.get('startTime'),
        'logs_url': build.get('logs', {}).get('deepLink')
    }


def stop_build(build_id: str) -> bool:
    """Stop a running build."""
    client = boto3.client('codebuild')

    try:
        client.stop_build(id=build_id)
        return True
    except client.exceptions.InvalidInputException:
        return False
```

4.3 IAM Role Configuration (Terraform)

Following existing Terraform patterns using `aws_iam_policy_document` data sources:

```
None
# Trust policy for CodeBuild service
data "aws_iam_policy_document" "codebuild_assume" {
  statement {
    effect = "Allow"

    principals {
      type        = "Service"
      identifiers = ["codebuild.amazonaws.com"]
    }

    actions = ["sts:AssumeRole"]
  }
}

# Create the IAM role for CodeBuild
resource "aws_iam_role" "codebuild" {
  name               = "${var.cluster_name}-codebuild-role"
  assume_role_policy = data.aws_iam_policy_document.codebuild_assume.json
```

```
  tags = {
    Name = "${var.cluster_name}-codebuild-role"
  }
}

# Permissions policy for CodeBuild
data "aws_iam_policy_document" "codebuild_permissions" {
  # S3 access to conversation paths
  statement {
    effect = "Allow"
    actions = [
      "s3:GetObject",
      "s3:GetObjectVersion",
      "s3:ListBucket"
    ]
    resources = [
      "arn:aws:s3:::aiconverterbucket",
      "arn:aws:s3:::aiconverterbucket/users/*"
    ]
  }

  # ECR permissions
  statement {
    effect = "Allow"
    actions = [
      "ecr:GetAuthorizationToken",
      "ecr:BatchCheckLayerAvailability",
      "ecr:GetDownloadUrlForLayer",
      "ecr:BatchGetImage",
      "ecr:PutImage",
      "ecr:InitiateLayerUpload",
      "ecr:UploadLayerPart",
      "ecr:CompleteLayerUpload"
    ]
    resources = ["*"]
  }

  # EKS permissions
  statement {
    effect = "Allow"
    actions = [
      "eks:DescribeCluster",
      "eks:ListClusters"
    ]
```

```
    resources = ["*"]
  }

  # CloudWatch Logs
  statement {
    effect = "Allow"
    actions = [
      "logs:CreateLogGroup",
      "logs:CreateLogStream",
      "logs:PutLogEvents"
    ]
    resources = ["arn:aws:logs:*:*:log-group:/aws/codebuild/*"]
  }
}

# Create the IAM policy
resource "aws_iam_policy" "codebuild" {
  name        = "${var.cluster_name}-codebuild-policy"
  description = "Policy for CodeBuild to access S3 conversations, ECR, and EKS"
  policy      = data.aws_iam_policy_document.codebuild_permissions.json

  tags = {
    Name = "${var.cluster_name}-codebuild-policy"
  }
}

# Attach policy to role
resource "aws_iam_role_policy_attachment" "codebuild" {
  role       = aws_iam_role.codebuild.name
  policy_arn = aws_iam_policy.codebuild.arn
}
```

---

5. Migration Path (CodeBuild to GitLab)

For sponsor documentation: The existing S3 structure (`users/{user}/conversations/{id}/generated/`) remains the source of truth. GitLab CI implementation requires:

1. **Config Extraction Job:** GitLab runner fetches `buildspec.yml` from conversation path using AWS CLI

2. **Translation Layer:** Convert buildspec commands to GitLab CI syntax before execution Or update system prompt to generate gitlab cli files based on template input.
3. **Storage:** Continue storing generated CI configs in existing conversation paths for audit history

---

Appendix A: References

- AWS CodeBuild S3 Source: https://docs.aws.amazon.com/codebuild/latest/userguide/create-project.html#create-project-source-s3
- CodeBuild start_build API (sourceLocationOverride): https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/codebuild/client/start_build.html
- Terraform IAM Policy Documents: https://registry.terraform.io/providers/hashicorp/aws/latest/docs/data-sources/iam_policy_document